INVERTIS UNIVERSITY BAREILLY

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS-553)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. RASHAN RAJ

SUBMMITED BY:

NISHANT TRIPATHI
ID: BCS2024554
ROLL NO: 2410303311

Contents

Lab Assignment	Page No.
Lab Assignment 1: Reading Array Values	3
Lab Assignment 2: Largest, Smallest and Count	9
Lab Assignment 3: Bubble Sort	15
Lab Assignment 4: Selection Sort	21
Lab Assignment 5: Linear Search	27
Lab Assignment 6: Binary Search on a Sorted Array	33
Lab Assignment 7: Practice Set	39
Lab Assignment 8: Classes and Objects	47
Lab Assignment 9: Constructors	53
Lab Assignment 10: Parent and Child Classes	59
Lab Assignment 11: Method Overriding	65
Lab Assignment 12: Recursive Factorial, Sum and Power	71
Lab Assignment 13: Recursion on Arrays	77
Lab Assignment 14: Sorted Check, Search and Fibonacci	83
Lab Assignment 15: Subsets and Arrangements	89

Lab Assignment 1: Reading Array Values

Experiment No. 1.1 - Read first and last array values

Experiment Title:

Read first and last array values

Aim:

To read the first and the last value of an array using indexes.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

An array stores values of the same type in consecutive indexed positions. Indexing starts at 0, so the first value is a[0] and the last is a[a.length - 1]. A for loop visits every index in turn, which lets a program read, print or add all the values.

Algorithm / Procedure:

1. Create the array marks = {45, 67, 80, 52, 90}.
2. Print marks[0] with the label First.
3. Print marks[marks.length - 1] with the label Last.
4. Compile and run the program.

Program / Queries / Commands:

```
public class Main {     public
static void main(String[] args) {
int[] marks = {45, 67, 80, 52, 90};

        System.out.println("First: " + marks[0]);
        System.out.println("Last: " + marks[marks.length - 1]);
    }
}
```

Input:

Fixed array declared in the program: marks = {45, 67, 80, 52, 90}.

Output:

```
First: 45
Last: 90
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. The first and last values of the array were read correctly using indexes.

Viva-Voce Questions:

- What is an array index?
- What is the first valid index in Java?
- How do you find the last index of an array?

Experiment No. 1.2 - Print all array values

Experiment Title:

Print all array values

Aim:

To print every value of an array on its own line using a for loop.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

An array stores values of the same type in consecutive indexed positions. Indexing starts at 0, so the first value is a[0] and the last is a[a.length - 1]. A for loop visits every index in turn, which lets a program read, print or add all the values.

Algorithm / Procedure:

1. Create the array marks = {45, 67, 80, 52, 90}.
2. Start a loop with i = 0 and continue while i < marks.length.
3. In each iteration print marks[i].
4. Compile and run the program.

Program / Queries / Commands:

```
public class Main {     public
static void main(String[]
args) {         int[] marks =
{45, 67, 80, 52, 90};

        for (int i = 0; i < marks.length; i++) {
System.out.println(marks[i]);
        }
    }
}
```

Input:

Fixed array declared in the program: marks = {45, 67, 80, 52, 90}.

Output:

```
45
67
80
```

```
52
90
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. All values of the array were printed in order using a for loop.

Viva-Voce Questions:

- What does marks.length return?
- Why does the loop start at 0?
- What is a loop iteration?

Experiment No. 1.3 - Calculate array sum

Experiment Title:

Calculate array sum.

Aim:

To add all the values of an array using a loop and print the total.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

An array stores values of the same type in consecutive indexed positions. Indexing starts at 0, so the first value is a[0] and the last is a[a.length - 1]. A for loop visits every index in turn, which lets a program read, print or add all the values.

Algorithm / Procedure:

1. Create the array and set sum = 0.
2. Visit every element with a for loop.
3. Add each value to sum.
4. Print sum with the label Sum.

Program / Queries / Commands:

```
public class Main {     public
static void main(String[]
args) {         int[] marks =
{45, 67, 80, 52, 90};
int sum = 0;

        for (int i = 0; i <
marks.length; i++) {
sum = sum + marks[i];         }

        System.out.println("Sum: " + sum);
    }
}
```

Input:

Fixed array declared in the program: marks = {45, 67, 80, 52, 90}.

Output:

```
Sum: 334
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The first and last values of the array were read correctly using indexes.

Viva-Voce Questions:

- What is an accumulator variable?
- Why is sum initialised to 0?
- What is the sum of the given array?

Lab Assignment 2: Largest, Smallest and Count

Experiment No. 2.1 - Find the largest value

Experiment Title:

Find the largest value

Aim:

To find the largest value in an array with a loop.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

To find the largest value, assume the first element is the largest and compare it with every other element, updating when a bigger value is found. The smallest value is found the same way with the comparison reversed. Counting uses a counter that is increased each time an element satisfies a condition.

Algorithm / Procedure:

1. Set largest = marks[0].
2. Loop from index 1 to the end.
3. If marks[i] > largest, set largest = marks[i].
4. Print largest with the label Largest.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int largest = marks[0];

        for (int i = 1; i < marks.length; i++) {
            if (marks[i] > largest) {
                largest = marks[i];
            }
        }
```

```
        System.out.println("Largest: " + largest);
    }
}
```

Input:

Fixed array declared in the program: marks = {45, 67, 80, 52, 90}.

Output:

```
Largest: 90
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The largest value of the array was found correctly.

Viva-Voce Questions:

- Why is largest initialised with marks[0]?
- Why does the loop start from index 1?
- What happens if all values are equal?

Experiment No. 2.2 - Find the smallest value

Experiment Title:

Find the smallest value

Aim:

To find the smallest value in an array with a loop.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

To find the largest value, assume the first element is the largest and compare it with every other element, updating when a bigger value is found. The smallest value is found the same way with the comparison reversed. Counting uses a counter that is increased each time an element satisfies a condition.

Algorithm / Procedure:

1. Set smallest = marks[0].
2. Loop from index 1 to the end.
3. If marks[i] < smallest, set smallest = marks[i].
4. Print smallest with the label Smallest.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int smallest = marks[0];

        for (int i = 1; i < marks.length; i++) {
            if (marks[i] < smallest) {
                smallest = marks[i];
            }
        }

        System.out.println("Smallest: " + smallest);
    }
}
```

Input:

Fixed array declared in the program: marks = {45, 67, 80, 52, 90}.

Output:

```
Smallest: 45
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The smallest value of the array was found correctly.

Viva-Voce Questions:

- How does this program differ from finding the largest value?
- What is the time complexity of finding the minimum?
- Can the smallest value be found without a loop?

Experiment No. 2.3 - Count values greater than 60

Experiment Title:

Count values greater than 60

Aim:

To count the array values that are greater than 60.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

To find the largest value, assume the first element is the largest and compare it with every other element, updating when a bigger value is found. The smallest value is found the same way with the comparison reversed. Counting uses a counter that is increased each time an element satisfies a condition.

Algorithm / Procedure:

1. Set count = 0.
2. Loop over every element.
3. If marks[i] > 60, increase count by 1.
4. Print count with the label Count.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int count = 0;

        for (int i = 0; i < marks.length; i++) {
            if (marks[i] > 60) {
                count++;
            }
        }

        System.out.println("Count: " + count);
    }
}
```

Input:

Fixed array declared in the program: marks = {45, 67, 80, 52, 90}.

Output:

```
Count: 3
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The number of values greater than 60 (3) was counted correctly.

Viva-Voce Questions:

- What does count++ do?
- Which values in the array are greater than 60?
- How would you count values less than 60?

Lab Assignment 3: Bubble Sort

Experiment No. 3.1 - One pass of bubble sort

Experiment Title:

One pass of bubble sort

Aim:

To perform a single pass of bubble sort on an array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Bubble sort repeatedly compares two neighbouring elements and swaps them if the left one is larger. After each pass the largest remaining value moves to its final position at the end, like a bubble rising to the surface. The sort needs an outer loop for the passes and an inner loop for the comparisons. Its worst-case time is $O(n^2)$.

Algorithm / Procedure:

1. Create the array {5, 1, 4, 2, 8}.
2. For j = 0 to length - 2 compare a[j] and a[j + 1].
3. If a[j] > a[j + 1], swap them using a temporary variable.
4. Print every value of the array.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};

        for (int j = 0; j < a.length - 1; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
```

```
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Input:

Fixed array declared in the program: {5, 1, 4, 2, 8}.

Output:

```
1
4
2
5
8
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. One pass moved the value 5 towards the end; the array after one pass is 1, 4, 2, 5, 8.

Viva-Voce Questions:

- What happens to the largest value after one pass?
- Why is a temporary variable needed for a swap?
- How many comparisons occur in one pass of n elements?

Experiment No. 3.2 - Bubble sort of {5, 1, 4, 2, 8}

Experiment Title:

Bubble sort of {5, 1, 4, 2, 8}

Aim:

To sort an array completely using bubble sort with an outer and an inner loop.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Bubble sort repeatedly compares two neighbouring elements and swaps them if the left one is larger. After each pass the largest remaining value moves to its final position at the end, like a bubble rising to the surface. The sort needs an outer loop for the passes and an inner loop for the comparisons. Its worst-case time is $O(n^2)$.

Algorithm / Procedure:

1. Create the array {5, 1, 4, 2, 8}.
2. Outer loop i runs for the passes (0 to length - 2).
3. Inner loop j compares a[j] and a[j + 1] up to length - 1 - i and swaps if out of order.
4. Print every value of the sorted array.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};

        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }

        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
```

```
}
```

Input:

Fixed array declared in the program: {5, 1, 4, 2, 8}.

Output:

```
1
2
4
5
8
```

Time / Space Complexity: Time $O(n^2)$ worst case; Space O(1).

Result:

The program was compiled and executed successfully. The array was sorted in ascending order: 1, 2, 4, 5, 8.

Viva-Voce Questions:

- What is the worst-case time complexity of bubble sort?
- Why does the inner loop run only up to length - 1 - i?
- Is bubble sort stable?

Experiment No. 3.3 - Bubble sort of {9, 7, 3}

Experiment Title:

Bubble sort of {9, 7, 3}

Aim:

To sort a small array in reverse order using bubble sort.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Bubble sort repeatedly compares two neighbouring elements and swaps them if the left one is larger. After each pass the largest remaining value moves to its final position at the end, like a bubble rising to the surface. The sort needs an outer loop for the passes and an inner loop for the comparisons. Its worst-case time is $O(n^2)$.

Algorithm / Procedure:

1. Create the array {9, 7, 3}.
2. Run the outer loop for the passes and the inner loop for the comparisons.
3. Swap neighbours when the left one is larger.
4. Print every value of the sorted array.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {9, 7, 3};

        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }

        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Input:

Fixed array declared in the program: {9, 7, 3}.

Output:

```
3
7
9
```

Time / Space Complexity: Time $O(n^2)$; Space O(1).

Result:

The program was compiled and executed successfully. The array {9, 7, 3} was sorted to 3, 7, 9.

Viva-Voce Questions:

- Which input is the worst case for bubble sort?
- How many swaps are needed for {9, 7, 3}?
- How can bubble sort be stopped early on a sorted array?

Lab Assignment 4: Selection Sort

Experiment No. 4.1 - Index of the smallest value

Experiment Title:

Index of the smallest value

Aim:

To find the index of the smallest value in an array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Selection sort divides the array into a sorted part and an unsorted part. In each step it finds the smallest value of the unsorted part and swaps it into the first unsorted position. It makes at most one swap per pass, but always needs $O(n^2)$ comparisons.

Algorithm / Procedure:

1. Create the array {64, 25, 12, 22, 11} and set minIndex = 0.
2. Loop from index 1 to the end.
3. If a[i] < a[minIndex], set minIndex = i.
4. Print minIndex.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};
        int minIndex = 0;

        for (int i = 1; i < a.length; i++) {
            if (a[i] < a[minIndex]) {
                minIndex = i;
            }
        }

        System.out.println(minIndex);
    }
}
```

Input:

Fixed array declared in the program: {64, 25, 12, 22, 11}.

Output:

```
4
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The smallest value 11 was located at index 4.

Viva-Voce Questions:

- Why do we store the index instead of the value?
- What is the role of minIndex?
- What is the time complexity of finding the minimum?

Experiment No. 4.2 - Selection sort

Experiment Title:

Selection sort

Aim:

To sort an array using the selection sort algorithm.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Selection sort divides the array into a sorted part and an unsorted part. In each step it finds the smallest value of the unsorted part and swaps it into the first unsorted position. It makes at most one swap per pass, but always needs $O(n^2)$ comparisons.

Algorithm / Procedure:

1. Create the array {64, 25, 12, 22, 11}.
2. For each position i, assume minIndex = i.
3. Scan positions i + 1 to the end and update minIndex when a smaller value is found.
4. Swap a[i] with a[minIndex].
5. Print every value of the sorted array.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};

        for (int i = 0; i < a.length - 1; i++) {
            int minIndex = i;

            for (int j = i + 1; j < a.length; j++) {
                if (a[j] < a[minIndex]) {
                    minIndex = j;
                }
            }

            int temp = a[i];
            a[i] = a[minIndex];
            a[minIndex] = temp;
        }
```

```
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Input:

Fixed array declared in the program: {64, 25, 12, 22, 11}.

Output:

```
11
12
22
25
64
```

Time / Space Complexity: Time $O(n^2)$ in all cases; Space O(1).

Result:

The program was compiled and executed successfully. The array was sorted in ascending order: 11, 12, 22, 25, 64.

Viva-Voce Questions:

- What is the time complexity of selection sort?
- How many swaps does selection sort make per pass?
- How does selection sort differ from bubble sort?

Experiment No. 4.3 - First step of selection sort

Experiment Title:

First step of selection sort

Aim:

To perform only the first step of selection sort.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Selection sort divides the array into a sorted part and an unsorted part. In each step it finds the smallest value of the unsorted part and swaps it into the first unsorted position. It makes at most one swap per pass, but always needs $O(n^2)$ comparisons.

Algorithm / Procedure:

1. Create the array and set minIndex = 0.
2. Find the index of the smallest value with a loop.
3. Swap a[0] with a[minIndex].
4. Print every value of the array.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};

        int minIndex = 0;

        for (int i = 1; i < a.length; i++) {
            if (a[i] < a[minIndex]) {
                minIndex = i;
            }
        }

        int temp = a[0];
        a[0] = a[minIndex];
        a[minIndex] = temp;

        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
```

```
    }
}
```

Input:

Fixed array declared in the program: {64, 25, 12, 22, 11}.

Output:

```
11
25
12
22
64
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. After the first step the array is 11, 25, 12, 22, 64.

Viva-Voce Questions:

- After the first step, which part of the array is sorted?
- Why is the swap needed?
- How many such steps are needed for n elements?

Lab Assignment 5: Linear Search

Experiment No. 5.1 - Linear search for a key that is present

Experiment Title:

Linear search for a key that is present

Aim:

To search for the key 80 in an array using linear search.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Linear search checks the elements one by one from the first to the last. If the key matches an element, its index is returned; if the end is reached without a match, -1 is returned. It works on unsorted arrays and takes O(n) time in the worst case.

Algorithm / Procedure:

1. Write linearSearch(a, key) that loops over every index.
2. If a[i] == key, return i.
3. After the loop, return -1.
4. Call it with key 80 and print the result.

Program / Queries / Commands:

```
public class Main {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == key) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(linearSearch(a, 80));
```

```
    }
}
```

Input:

Array {45, 67, 80, 52, 90}, key = 80.

Output:

```
2
```

Time / Space Complexity: Time O(n), best case O(1); Space O(1).

Result:

The program was compiled and executed successfully. The key 80 was found at index 2.

Viva-Voce Questions:

- What does linearSearch return when the key is missing?
- Does linear search need a sorted array?
- What is the best case of linear search?

Experiment No. 5.2 - Linear search for a key that is absent

Experiment Title:

Linear search for a key that is absent

Aim:

To show that linear search returns -1 when the key is not in the array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Linear search checks the elements one by one from the first to the last. If the key matches an element, its index is returned; if the end is reached without a match, -1 is returned. It works on unsorted arrays and takes O(n) time in the worst case.

Algorithm / Procedure:

1. Write linearSearch(a, key) that loops over every index.
2. If a[i] == key, return i.
3. After the loop ends without a match, return -1.
4. Call it with key 100 and print the result.

Program / Queries / Commands:

```
public class Main {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == key) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(linearSearch(a, 100));
    }
}
```

Input:

Array {45, 67, 80, 52, 90}, key = 100.

Output:

```
-1
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The key 100 is not in the array, so the method returned -1.

Viva-Voce Questions:

- Why is -1 used for 'not found'?
- How many comparisons are made for an absent key?
- What is the worst case of linear search?

Experiment No. 5.3 - Count occurrences of a value

Experiment Title:

Count occurrences of a value

Aim:

To count how many times the value 52 appears in an array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Linear search checks the elements one by one from the first to the last. If the key matches an element, its index is returned; if the end is reached without a match, -1 is returned. It works on unsorted arrays and takes O(n) time in the worst case.

Algorithm / Procedure:

1. Create the array {52, 10, 52, 7, 52} and set count = 0.
2. Loop over every element.
3. If a[i] == 52, increase count.
4. Print count with the label Count.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {52, 10, 52, 7, 52};
        int count = 0;

        for (int i = 0; i < a.length; i++) {
            if (a[i] == 52) {
                count++;
            }
        }

        System.out.println("Count: " + count);
    }
}
```

Input:

Array {52, 10, 52, 7, 52}, value = 52.

Output:

```
Count: 3
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The value 52 appears 3 times in the array.

Viva-Voce Questions:

- How is counting different from searching for the first match?
- Why must the loop visit every element?
- What is the time complexity?

Lab Assignment 6: Binary Search on a Sorted Array

Experiment No. 6.1 - Binary search for key 40

Experiment Title:

Binary search for key 40

Aim:

To search for the key 40 in a sorted array using binary search.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Binary search works only on a sorted array. It compares the key with the middle element; if the key is smaller it continues in the left half, if larger in the right half. Because the search range halves every step, the time complexity is O(log n).

Algorithm / Procedure:

1. Set low = 0 and high = length - 1.
2. While low <= high, compute mid = low + (high - low) / 2.
3. If a[mid] == key return mid; if a[mid] < key set low = mid + 1; otherwise set high = mid - 1.
4. If the loop ends, return -1.

Program / Queries / Commands:

```
public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0;
        int high = a.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
```

```
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 40));
    }
}
```

Input:

Sorted array {10, 20, 30, 40, 50}, key = 40.

Output:

```
3
```

Time / Space Complexity: Time O(log n), best case O(1); Space O(1).

Result:

The program was compiled and executed successfully. The key 40 was found at index 3.

Viva-Voce Questions:

- Why must the array be sorted for binary search?
- What is the time complexity of binary search?
- Why is mid computed as low + (high - low) / 2?

Experiment No. 6.2 - Binary search for key 35 (absent)

Experiment Title:

Binary search for key 35 (absent)

Aim:

To show that binary search returns -1 when the key is not present.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Binary search works only on a sorted array. It compares the key with the middle element; if the key is smaller it continues in the left half, if larger in the right half. Because the search range halves every step, the time complexity is O(log n).

Algorithm / Procedure:

1. Set low = 0 and high = length - 1.
2. While low <= high, compute mid = low + (high - low) / 2.
3. If a[mid] == key return mid; if a[mid] < key set low = mid + 1; otherwise set high = mid - 1.
4. If the loop ends, return -1.

Program / Queries / Commands:

```
public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0;
        int high = a.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }
```

```
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 35));
    }
}
```

Input:

Sorted array {10, 20, 30, 40, 50}, key = 35.

Output:

```
-1
```

Time / Space Complexity: Time O(log n); Space O(1).

Result:

The program was compiled and executed successfully. The key 35 is not in the array, so the method returned -1.

Viva-Voce Questions:

- When does the while loop stop for an absent key?
- How many steps are needed for 35 here?
- What is returned for a missing key?

Experiment No. 6.3 - Binary search for key 10 (first element)

Experiment Title:

Binary search for key 10 (first element)

Aim:

To search for the first element of a sorted array using binary search.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Binary search works only on a sorted array. It compares the key with the middle element; if the key is smaller it continues in the left half, if larger in the right half. Because the search range halves every step, the time complexity is O(log n).

Algorithm / Procedure:

1. Set low = 0 and high = length - 1.
2. While low <= high, compute mid = low + (high - low) / 2.
3. If a[mid] == key return mid; if a[mid] < key set low = mid + 1; otherwise set high = mid - 1.
4. If the loop ends, return -1.

Program / Queries / Commands:

```
public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0;
        int high = a.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }
```

```
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 10));
    }
}
```

Input:

Sorted array {10, 20, 30, 40, 50}, key = 10.

Output:

```
0
```

Time / Space Complexity: Time O(log n); Space O(1).

Result:

The program was compiled and executed successfully. The key 10 was found at index 0.

Viva-Voce Questions:

- How does the search range change for key 10?
- Which is faster for large arrays: linear or binary search?
- Can binary search be written recursively?

Lab Assignment 7: Practice Set

Experiment No. 7.1 - Search for 52 with a loop

Experiment Title:

Search for 52 with a loop

Aim:

To search for the value 52 in an array and print its index or 'Not found'.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

This practice set combines the earlier ideas: scanning an array with a loop to search, sorting with bubble sort, accumulating a product with a loop, and filtering values with an if statement. A value is even when value % 2 is 0.

Algorithm / Procedure:

1. Set index = -1.
2. Loop over the array; if a[i] == 52, store i in index and break.
3. If index != -1 print Found at index followed by the index.
4. Otherwise print Not found.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {45, 52, 80};
        int index = -1;

        for (int i = 0; i < a.length; i++) {
            if (a[i] == 52) {
                index = i;
                break;
            }
        }

        if (index != -1) {
```

```
            System.out.println("Found at index " + index);
        } else {
            System.out.println("Not found");
        }
    }
}
```

Input:

Array {45, 52, 80}, value = 52.

Output:

```
Found at index 1
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The value 52 was found at index 1.

Viva-Voce Questions:

- What does the break statement do?
- Why is index initialised to -1?
- What is printed if the value is absent?

Experiment No. 7.2 - Bubble sort of {3, 1, 2}

Experiment Title:

Bubble sort of {3, 1, 2}

Aim:

To sort the array {3, 1, 2} using bubble sort.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

This practice set combines the earlier ideas: scanning an array with a loop to search, sorting with bubble sort, accumulating a product with a loop, and filtering values with an if statement. A value is even when value % 2 is 0.

Algorithm / Procedure:

1. Create the array {3, 1, 2}.
2. Use an outer loop for the passes and an inner loop for the comparisons.
3. Swap neighbours when the left one is larger.
4. Print every value of the sorted array.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {3, 1, 2};

        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }

        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Input:

Fixed array declared in the program: {3, 1, 2}.

Output:

```
1
2
3
```

Time / Space Complexity: Time $O(n^2)$; Space O(1).

Result:

The program was compiled and executed successfully. The array was sorted in ascending order: 1, 2, 3.

Viva-Voce Questions:

- How many passes are needed for 3 elements?
- What is the time complexity of bubble sort?
- Why is a temporary variable used?

Experiment No. 7.3 - Product of array values

Experiment Title:

Product of array values

Aim:

To multiply all the values of an array using a for loop.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

This practice set combines the earlier ideas: scanning an array with a loop to search, sorting with bubble sort, accumulating a product with a loop, and filtering values with an if statement. A value is even when value % 2 is 0.

Algorithm / Procedure:

1. Create the array {2, 3, 4} and set product = 1.
2. Loop over every element.
3. Multiply product by each value.
4. Print product with the label Product.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {2, 3, 4};
        int product = 1;

        for (int i = 0; i < a.length; i++) {
            product = product * a[i];
        }

        System.out.println("Product: " + product);
    }
}
```

Input:

Fixed array declared in the program: {2, 3, 4}.

Output:

```
Product: 24
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The product of the values (24) was calculated correctly.

Viva-Voce Questions:

- Why is product initialised to 1 and not 0?
- What would happen if it started at 0?
- What is the product of the given array?

Experiment No. 7.4 - Print even values

Experiment Title:

Print even values

Aim:

To print each even value of an array using a loop and an if statement.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

This practice set combines the earlier ideas: scanning an array with a loop to search, sorting with bubble sort, accumulating a product with a loop, and filtering values with an if statement. A value is even when value % 2 is 0.

Algorithm / Procedure:

1. Create the array {1, 2, 3, 4}.
2. Loop over every element.
3. If a[i] % 2 == 0, print a[i].
4. Skip odd values.

Program / Queries / Commands:

```
public class Main {
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4};

        for (int i = 0; i < a.length; i++) {
            if (a[i] % 2 == 0) {
                System.out.println(a[i]);
            }
        }
    }
}
```

Input:

Fixed array declared in the program: {1, 2, 3, 4}.

Output:

```
2
4
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. The even values 2 and 4 were printed.

Viva-Voce Questions:

- What does the % operator do?
- How do you test whether a number is odd?
- What is the output for {1, 2, 3, 4}?

Lab Assignment 8: Classes and Objects

Experiment No. 8.1 - Class Student with one object

Experiment Title:

Class Student with one object

Aim:

To create a class Student and one object, and call its show() method.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

A class is a blueprint that describes the fields (data) and methods (behaviour) of a kind of object. An object is a real instance created from the class with the new keyword. Many objects can be created from one class, and each has its own copy of the fields. A method with return type boolean returns true or false.

Algorithm / Procedure:

1. Define class Student with fields roll and name and method show().
2. In main, create an object with new Student().
3. Set roll = 5 and name = Rohan.
4. Call show().

Program / Queries / Commands:

```
class Student {
    int roll;
    String name;

    void show() {
        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.roll = 5;
        s.name = "Rohan";
        s.show();
    }
}
```

Input:

Roll = 5, Name = Rohan.

Output:

```
Roll: 5
Name: Rohan
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. The Student object was created and its details were displayed.

Viva-Voce Questions:

- What is the difference between a class and an object?
- What does the new keyword do?
- What is a field?

Experiment No. 8.2 - Class Student with two objects

Experiment Title:

Class Student with two objects

Aim:

To create two independent objects of the same class.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

A class is a blueprint that describes the fields (data) and methods (behaviour) of a kind of object. An object is a real instance created from the class with the new keyword. Many objects can be created from one class, and each has its own copy of the fields. A method with return type boolean returns true or false.

Algorithm / Procedure:

1. Define class Student with fields roll and name and method show().
2. Create objects s1 and s2 with new Student().
3. Set s1 to (5, Rohan) and s2 to (8, Meera).
4. Call show() on each object.

Program / Queries / Commands:

```
class Student {
    int roll;
    String name;

    void show() {
        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 5;
        s1.name = "Rohan";

        Student s2 = new Student();
```

```
        s2.roll = 8;
        s2.name = "Meera";

        s1.show();
        s2.show();
    }
}
```

Input:

Student 1: Roll 5, Rohan. Student 2: Roll 8, Meera.

Output:

```
Roll: 5
Name: Rohan
Roll: 8
Name: Meera
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. Two objects were created and each kept its own field values.

Viva-Voce Questions:

- Does changing s1 affect s2?
- How many objects can one class create?
- What is a reference variable?

Experiment No. 8.3 - Class Book with isLong()

Experiment Title:

Class Book with isLong()

Aim:

To write a method that returns a boolean based on a field value.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

A class is a blueprint that describes the fields (data) and methods (behaviour) of a kind of object. An object is a real instance created from the class with the new keyword. Many objects can be created from one class, and each has its own copy of the fields. A method with return type boolean returns true or false.

Algorithm / Procedure:

1. Define class Book with field pages.
2. Write boolean isLong() that returns pages > 300.
3. Create two Book objects with 450 and 120 pages.
4. Print isLong() of each object.

Program / Queries / Commands:

```
class Book {
    int pages;

    boolean isLong() {
        return pages > 300;
    }
}

public class Main {
    public static void main(String[] args) {
        Book b1 = new Book();
        b1.pages = 450;

        Book b2 = new Book();
        b2.pages = 120;

        System.out.println(b1.isLong());
```

```
        System.out.println(b2.isLong());
    }
}
```

Input:

Book 1: 450 pages. Book 2: 120 pages.

Output:

```
true
false
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. isLong() returned true for 450 pages and false for 120 pages.

Viva-Voce Questions:

- What does a boolean method return?
- How does return pages > 300 work?
- Can two objects give different results from the same method?

Lab Assignment 9: Constructors

Experiment No. 9.1 - Constructor with no parameters

Experiment Title:

Constructor with no parameters

Aim:

To set a field value using a constructor with no parameters.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

A constructor is a special method that runs automatically when an object is created with new. Its name is the same as the class name and it has no return type. A constructor with parameters lets the caller supply the initial values. The keyword this refers to the current object, so this.width is the field while width alone is the parameter.

Algorithm / Procedure:

1. Define class Box with field width.
2. Write a no-argument constructor Box() that sets width = 10.
3. Create an object with new Box().
4. Print the width.

Program / Queries / Commands:

```
class Box {
    int width;

    Box() {
        width = 10;
    }
}

public class Main {
    public static void main(String[] args) {
        Box b = new Box();
```

```
        System.out.println("Width: " + b.width);
    }
}
```

Input:

No input; the constructor sets width = 10.

Output:

```
Width: 10
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. The constructor initialised width to 10 when the object was created.

Viva-Voce Questions:

- What is a constructor?
- Does a constructor have a return type?
- When does a constructor run?

Experiment No. 9.2 - Constructor with a parameter

Experiment Title:

Constructor with a parameter

Aim:

To set a field using a parameterised constructor and the this keyword.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

A constructor is a special method that runs automatically when an object is created with new. Its name is the same as the class name and it has no return type. A constructor with parameters lets the caller supply the initial values. The keyword this refers to the current object, so this.width is the field while width alone is the parameter.

Algorithm / Procedure:

1. Define class Box with field width.
2. Write constructor Box(int width) that sets this.width = width.
3. Create an object with new Box(25).
4. Print the width.

Program / Queries / Commands:

```
class Box {
    int width;

    Box(int width) {
        this.width = width;
    }
}

public class Main {
    public static void main(String[] args) {
        Box b = new Box(25);
        System.out.println("Width: " + b.width);
    }
}
```

Input:

Constructor argument: 25.

Output:

```
Width: 25
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. The parameterised constructor set width to 25.

Viva-Voce Questions:

- What does the this keyword refer to?
- Why is this.width = width needed?
- What is a parameterised constructor?

Experiment No. 9.3 - Class Counter

Experiment Title:

Class Counter

Aim:

To use a constructor to start a counter at 0 and a method to increase it.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

A constructor is a special method that runs automatically when an object is created with new. Its name is the same as the class name and it has no return type. A constructor with parameters lets the caller supply the initial values. The keyword this refers to the current object, so this.width is the field while width alone is the parameter.

Algorithm / Procedure:

1. Define class Counter with field count.
2. Constructor sets count = 0.
3. increment() adds 1 to count.
4. Create one Counter, call increment() three times, print count.

Program / Queries / Commands:

```
class Counter {
    int count;

    Counter() {
        count = 0;
    }

    void increment() {
        count = count + 1;
    }
}

public class Main {
    public static void main(String[] args) {
        Counter c = new Counter();
        c.increment();
        c.increment();
```

```
        c.increment();
        System.out.println("Count: " + c.count);
    }
}
```

Input:

No input; increment() is called 3 times.

Output:

```
Count: 3
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. The counter started at 0 and reached 3 after three increments.

Viva-Voce Questions:

- What value does count have after construction?
- What happens if no constructor is written?
- How does increment() change the object?

Lab Assignment 10: Parent and Child Classes

Experiment No. 10.1 - Animal and Dog

Experiment Title:

Animal and Dog

Aim:

To show that a child class can use a method of its parent class.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Inheritance lets a child (sub) class acquire the fields and methods of a parent (super) class using the extends keyword, which avoids rewriting code. When the parent has a constructor with parameters, the child constructor must call it with super(...) as its first statement.

Algorithm / Procedure:

1. Define class Animal with method sound().
2. Define class Dog extends Animal with an empty body.
3. Create a Dog object.
4. Call sound() on it.

Program / Queries / Commands:

```
class Animal {
    void sound() {
        System.out.println("Some sound");
    }
}

class Dog extends Animal {
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
```

```
    }
}
```

Input:

No input.

Output:

```
Some sound
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. Dog used the sound() method inherited from Animal.

Viva-Voce Questions:

- What is inheritance?
- Which keyword is used for inheritance in Java?
- What are superclass and subclass?

Experiment No. 10.2 - Person and Student with super()

Experiment Title:

Person and Student with super()

Aim:

To call the parent constructor from the child constructor using super(name).

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Inheritance lets a child (sub) class acquire the fields and methods of a parent (super) class using the extends keyword, which avoids rewriting code. When the parent has a constructor with parameters, the child constructor must call it with super(...) as its first statement.

Algorithm / Procedure:

1. Define class Person with field name and constructor Person(String name).
2. Define class Student extends Person with constructor Student(String name) calling super(name).
3. Create new Student("Neha").
4. Print the name.

Program / Queries / Commands:

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

class Student extends Person {
    Student(String name) {
        super(name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Neha");
```

```
        System.out.println("Name: " + s.name);
    }
}
```

Input:

Student name: Neha.

Output:

```
Name: Neha
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. The Student constructor passed the name to Person using super(name).

Viva-Voce Questions:

- What does super(name) do?
- Why must super(...) be the first statement?
- Can a child class access the parent's fields?

Experiment No. 10.3 - Person and Teacher

Experiment Title:

Person and Teacher

Aim:

To show that a child class inherits both fields and methods of its parent.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Inheritance lets a child (sub) class acquire the fields and methods of a parent (super) class using the extends keyword, which avoids rewriting code. When the parent has a constructor with parameters, the child constructor must call it with super(...) as its first statement.

Algorithm / Procedure:

1. Define class Person with field name and method greet().
2. Define class Teacher extends Person with an empty body.
3. Create a Teacher and set name = Kiran.
4. Call greet().

Program / Queries / Commands:

```
class Person {
    String name;

    void greet() {
        System.out.println("Hello, " + name);
    }
}

class Teacher extends Person {
}

public class Main {
    public static void main(String[] args) {
        Teacher t = new Teacher();
        t.name = "Kiran";
        t.greet();
    }
}
```

Input:

Teacher name: Kiran.

Output:

```
Hello, Kiran
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. Teacher used the inherited name field and greet() method.

Viva-Voce Questions:

- What is code reusability?
- Does Teacher need its own greet() method?
- What is the IS-A relationship?

Lab Assignment 11: Method Overriding

Experiment No. 11.1 - Shape and Square

Experiment Title:

Shape and Square

Aim:

To override area() in a Square class.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Method overriding means a child class provides its own version of a method that already exists in the parent class, with the same name and parameters. When the method is called on a child object, the child's version runs. A parent-type variable that holds a child object also runs the child's version; this is runtime polymorphism.

Algorithm / Procedure:

1. Define class Shape with area() returning 0.
2. Define Square extends Shape with field side and area() returning side * side.
3. Create a Square with side = 4.
4. Print Area with the result of area().

Program / Queries / Commands:

```
class Shape {
    int area() {
        return 0;
    }
}

class Square extends Shape {
    int side;

    @Override
```

```
    int area() {
        return side * side;
    }
}

public class Main {
    public static void main(String[] args) {
        Square s = new Square();
        s.side = 4;
        System.out.println("Area: " + s.area());
    }
}
```

Input:

Square side = 4.

Output:

```
Area: 16
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. Square.area() overrode Shape.area() and returned 16.

Viva-Voce Questions:

- What is method overriding?
- What does the @Override annotation do?
- What is the difference between overriding and overloading?

Experiment No. 11.2 - Shape and Rectangle

Experiment Title:

Shape and Rectangle

Aim:

To override area() in a Rectangle class.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Method overriding means a child class provides its own version of a method that already exists in the parent class, with the same name and parameters. When the method is called on a child object, the child's version runs. A parent-type variable that holds a child object also runs the child's version; this is runtime polymorphism.

Algorithm / Procedure:

1. Define class Shape with area() returning 0.
2. Define Rectangle extends Shape with fields length and breadth and area() returning length * breadth.
3. Create a Rectangle with length 4 and breadth 5.
4. Print Area with the result of area().

Program / Queries / Commands:

```
class Shape {
    int area() {
        return 0;
    }
}

class Rectangle extends Shape {
    int length;
    int breadth;

    @Override
    int area() {
        return length * breadth;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = 4;
        r.breadth = 5;
        System.out.println("Area: " + r.area());
    }
}
```

Input:

Rectangle length = 4, breadth = 5.

Output:

```
Area: 20
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. Rectangle.area() overrode Shape.area() and returned 20.

Viva-Voce Questions:

- Which version of area() runs for a Rectangle object?
- Can a child override a parent method with different parameters?
- What is the default area() of Shape?

Experiment No. 11.3 - Parent-type variable holding a Square

Experiment Title:

Parent-type variable holding a Square

Aim:

To demonstrate runtime polymorphism using a Shape variable that holds a Square object.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Method overriding means a child class provides its own version of a method that already exists in the parent class, with the same name and parameters. When the method is called on a child object, the child's version runs. A parent-type variable that holds a child object also runs the child's version; this is runtime polymorphism.

Algorithm / Procedure:

1. Define Shape and Square as before.
2. Create a Square with side = 3.
3. Store it in a variable of type Shape.
4. Call area() on the Shape variable and print the result.

Program / Queries / Commands:

```
class Shape {
    int area() {
        return 0;
    }
}

class Square extends Shape {
    int side;

    @Override
    int area() {
        return side * side;
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        Square sq = new Square();
        sq.side = 3;

        Shape s = sq;
        System.out.println("Area: " + s.area());
    }
}
```

Input:

Square side = 3.

Output:

```
Area: 9
```

Time / Space Complexity: Time O(1); Space O(1).

Result:

The program was compiled and executed successfully. The Shape variable called Square's area() and printed 9, showing runtime polymorphism.

Viva-Voce Questions:

- What is polymorphism?
- Which area() runs when a Shape variable holds a Square?
- Why can a Shape variable not access the field side?

Lab Assignment 12: Recursive Factorial, Sum and Power

Experiment No. 12.1 - Recursive factorial

Experiment Title:

Recursive factorial

Aim:

To compute the factorial of a number using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion is a technique in which a method calls itself on a smaller input. Every recursive method needs a base case, which stops the calls, and a recursive case, which reduces the problem. Each call is stored on the call stack, so the space used grows with the depth of the recursion. Without a base case the program ends with StackOverflowError.

Algorithm / Procedure:

1. Base case: if n == 0 return 1.
2. Recursive case: return n * factorial(n - 1).
3. Call factorial(5) from main and print the result.

Program / Queries / Commands:

```
public class Main {
    static int factorial(int n) {
        if (n == 0) {
            return 1;
        }
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

Input:

n = 5.

Output:

```
120
```

Time / Space Complexity: Time O(n); Space O(n) for the call stack.

Result:

The program was compiled and executed successfully. factorial(5) was computed recursively and equals 120.

Viva-Voce Questions:

- What is recursion?
- What is a base case?
- What is the factorial of 0?

Experiment No. 12.2 - Recursive sum of 1 to n

Experiment Title:

Recursive sum of 1 to n

Aim:

To add the numbers from 1 to n using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion is a technique in which a method calls itself on a smaller input. Every recursive method needs a base case, which stops the calls, and a recursive case, which reduces the problem. Each call is stored on the call stack, so the space used grows with the depth of the recursion. Without a base case the program ends with StackOverflowError.

Algorithm / Procedure:

1. Base case: if n == 0 return 0.
2. Recursive case: return n + sum(n - 1).
3. Call sum(10) from main and print the result.

Program / Queries / Commands:

```
public class Main {
    static int sum(int n) {
        if (n == 0) {
            return 0;
        }
        return n + sum(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(sum(10));
    }
}
```

Input:

n = 10.

Output:

```
55
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. sum(10) was computed recursively and equals 55.

Viva-Voce Questions:

- Why is the base case 0 here?
- What happens if the base case is missing?
- How can sum(n) be written without recursion?

Experiment No. 12.3 - Recursive power

Experiment Title:

Recursive power

Aim:

To compute base raised to exp using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion is a technique in which a method calls itself on a smaller input. Every recursive method needs a base case, which stops the calls, and a recursive case, which reduces the problem. Each call is stored on the call stack, so the space used grows with the depth of the recursion. Without a base case the program ends with StackOverflowError.

Algorithm / Procedure:

1. Base case: if exp == 0 return 1.
2. Recursive case: return base * power(base, exp - 1).
3. Call power(2, 5) from main and print the result.

Program / Queries / Commands:

```
public class Main {
    static int power(int base, int exp) {
        if (exp == 0) {
            return 1;
        }
        return base * power(base, exp - 1);
    }

    public static void main(String[] args) {
        System.out.println(power(2, 5));
    }
}
```

Input:

base = 2, exp = 5.

Output:

```
32
```

Time / Space Complexity: Time O(exp); Space O(exp).

Result:

The program was compiled and executed successfully. power(2, 5) was computed recursively and equals 32.

Viva-Voce Questions:

- What is the base case of power()?
- How many recursive calls are made for exp = 5?
- What is the call stack?

Lab Assignment 13: Recursion on Arrays

Experiment No. 13.1 - Recursive array sum

Experiment Title:

Recursive array sum

Aim:

To find the sum of array values from index i to the end using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

To apply recursion to an array, an extra parameter i holds the current index. Each call handles a[i] and then calls itself with i + 1 (or i - 1) for the rest of the array. The base case is reached when the index goes past the end (or before the start) of the array, or when only one element remains.

Algorithm / Procedure:

1. Base case: if i == a.length return 0.
2. Recursive case: return a[i] + sum(a, i + 1).
3. Call sum(a, 0) from main and print the result.

Program / Queries / Commands:

```
public class Main {
    static int sum(int[] a, int i) {
        if (i == a.length) {
            return 0;
        }
        return a[i] + sum(a, i + 1);
    }

    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        System.out.println(sum(a, 0));
    }
}
```

Input:

Array {3, 6, 9, 12, 15}, i = 0.

Output:

```
45
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. The recursive sum of the array is 45.

Viva-Voce Questions:

- Why is the index passed as a parameter?
- What is the base case here?
- How deep is the recursion for n elements?

Experiment No. 13.2 - Recursive array maximum

Experiment Title:

Recursive array maximum

Aim:

To find the largest value from index i to the end using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

To apply recursion to an array, an extra parameter i holds the current index. Each call handles a[i] and then calls itself with i + 1 (or i - 1) for the rest of the array. The base case is reached when the index goes past the end (or before the start) of the array, or when only one element remains.

Algorithm / Procedure:

1. Base case: if i is the last index, return a[i].
2. Find rest = max(a, i + 1).
3. Return the larger of a[i] and rest.
4. Call max(a, 0) from main and print the result.

Program / Queries / Commands:

```
public class Main {
    static int max(int[] a, int i) {
        if (i == a.length - 1) {
            return a[i];
        }
        int rest = max(a, i + 1);
        if (a[i] > rest) {
            return a[i];
        }
        return rest;
    }

    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        System.out.println(max(a, 0));
    }
}
```

Input:

Array {3, 6, 9, 12, 15}, i = 0.

Output:

```
15
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. The recursive maximum of the array is 15.

Viva-Voce Questions:

- Why is the base case the last index and not a.length?
- How does the method compare values?
- What is the time complexity?

Experiment No. 13.3 - Print an array in reverse

Experiment Title:

Print an array in reverse

Aim:

To print array values from index i down to 0 using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

To apply recursion to an array, an extra parameter i holds the current index. Each call handles a[i] and then calls itself with i + 1 (or i - 1) for the rest of the array. The base case is reached when the index goes past the end (or before the start) of the array, or when only one element remains.

Algorithm / Procedure:

1. Base case: if i < 0 return.
2. Print a[i].
3. Call printReverse(a, i - 1).
4. Call it from main with i = a.length - 1.

Program / Queries / Commands:

```
public class Main {
    static void printReverse(int[] a, int i) {
        if (i < 0) {
            return;
        }
        System.out.println(a[i]);
        printReverse(a, i - 1);
    }

    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        printReverse(a, a.length - 1);
    }
}
```

Input:

Array {3, 6, 9, 12, 15}, i = 4.

Output:

```
15
12
9
6
3
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. The array was printed in reverse order: 15, 12, 9, 6, 3.

Viva-Voce Questions:

- What does a void method return?
- What changes if the print is placed after the recursive call?
- What is the base case?

Lab Assignment 14: Sorted Check, Search and Fibonacci

Experiment No. 14.1 - Recursive sorted check

Experiment Title:

Recursive sorted check

Aim:

To check whether an array is in ascending order using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion can check a property of an array by examining one position and delegating the rest to the next call. A recursive search returns the index when a[i] matches the key and -1 when the index passes the end. The Fibonacci sequence is defined as fib(0) = 0, fib(1) = 1 and fib(n) = fib(n - 1) + fib(n - 2); it needs two base cases and makes two recursive calls, so the plain recursive version takes exponential time.

Algorithm / Procedure:

1. Base case: if i >= a.length - 1 return true.
2. If a[i] > a[i + 1] return false.
3. Otherwise return isSorted(a, i + 1).
4. Call it from main with i = 0 and print the result.

Program / Queries / Commands:

```
public class Main {
    static boolean isSorted(int[] a, int i) {
        if (i >= a.length - 1) {
            return true;
        }
        if (a[i] > a[i + 1]) {
            return false;
        }
        return isSorted(a, i + 1);
```

```
    }

    public static void main(String[] args) {
        int[] a = {2, 4, 6, 8, 10};
        System.out.println(isSorted(a, 0));
    }
}
```

Input:

Array {2, 4, 6, 8, 10}, i = 0.

Output:

```
true
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. The array is in ascending order, so the method returned true.

Viva-Voce Questions:

- Why is the base case a.length - 1?
- When does the method return false?
- What does the method return for a one-element array?

Experiment No. 14.2 - Recursive search by index

Experiment Title:

Recursive search by index

Aim:

To find the index of a key from index i onwards using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion can check a property of an array by examining one position and delegating the rest to the next call. A recursive search returns the index when a[i] matches the key and -1 when the index passes the end. The Fibonacci sequence is defined as fib(0) = 0, fib(1) = 1 and fib(n) = fib(n - 1) + fib(n - 2); it needs two base cases and makes two recursive calls, so the plain recursive version takes exponential time.

Algorithm / Procedure:

1. Base case: if i == a.length return -1.
2. If a[i] == key return i.
3. Otherwise return findIndex(a, key, i + 1).
4. Call it from main with key 80 and i = 0.

Program / Queries / Commands:

```
public class Main {
    static int findIndex(int[] a, int key, int i) {
        if (i == a.length) {
            return -1;
        }
        if (a[i] == key) {
            return i;
        }
        return findIndex(a, key, i + 1);
    }

    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(findIndex(a, 80, 0));
    }
}
```

Input:

Array {45, 67, 80, 52, 90}, key = 80, i = 0.

Output:

```
2
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. The key 80 was found at index 2 using recursion.

Viva-Voce Questions:

- How is this similar to linear search?
- What is returned when the key is absent?
- Why does recursion use more space than a loop?

Experiment No. 14.3 - Recursive Fibonacci

Experiment Title:

Recursive Fibonacci

Aim:

To compute the nth Fibonacci number using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion can check a property of an array by examining one position and delegating the rest to the next call. A recursive search returns the index when a[i] matches the key and -1 when the index passes the end. The Fibonacci sequence is defined as fib(0) = 0, fib(1) = 1 and fib(n) = fib(n - 1) + fib(n - 2); it needs two base cases and makes two recursive calls, so the plain recursive version takes exponential time.

Algorithm / Procedure:

1. Base cases: if n == 0 return 0; if n == 1 return 1.
2. Recursive case: return fib(n - 1) + fib(n - 2).
3. Call fib(7) from main and print the result.

Program / Queries / Commands:

```
public class Main {
    static int fib(int n) {
        if (n == 0) {
            return 0;
        }
        if (n == 1) {
            return 1;
        }
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        System.out.println(fib(7));
    }
}
```

Input:

n = 7.

Output:

```
13
```

Time / Space Complexity: Time $O(2^n)$; Space O(n).

Result:

The program was compiled and executed successfully. fib(7) was computed recursively and equals 13.

Viva-Voce Questions:

- Why does fib need two base cases?
- What is the time complexity of recursive Fibonacci?
- Why is the same value computed many times?

Lab Assignment 15: Subsets and Arrangements

Experiment No. 15.1 - All subsets of {1, 2, 3}

Experiment Title:

All subsets of {1, 2, 3}

Aim:

To print all subsets of an array using recursion and backtracking.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Backtracking builds a solution one choice at a time. After a complete answer is printed, the last choice is undone so that the next choice can be tried. For subsets, each element is either skipped or included, giving 2^n subsets. For arrangements (permutations), a used[] array marks letters already placed, giving n! arrangements.

Algorithm / Procedure:

1. Keep a list current for the subset being built; i is the current index.
2. Base case: when i == length, print current in braces.
3. First skip a[i]: call subsets(a, i + 1).
4. Then include a[i]: add it, call subsets(a, i + 1), and remove it (backtrack).

Program / Queries / Commands:

```
import java.util.ArrayList;

public class Main {
    static void subsets(int[] a, int i, ArrayList<Integer> current) {
        if (i == a.length) {
            System.out.print("{");
            for (int k = 0; k < current.size(); k++) {
                if (k > 0) {
                    System.out.print(", ");
                }
```

```
                System.out.print(current.get(k));
            }
            System.out.println("}");
            return;
        }


        subsets(a, i + 1, current);


        current.add(a[i]);
        subsets(a, i + 1, current);


        current.remove(current.size() - 1);
    }

    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        subsets(a, 0, new ArrayList<Integer>());
    }
}
```

Input:

Array {1, 2, 3}.

Output:

```
{}
{3}
{2}
{2, 3}
{1}
{1, 3}
{1, 2}
{1, 2, 3}
```

Time / Space Complexity: Time O(n * 2^n); Space O(n).

Result:

The program was compiled and executed successfully. All 8 subsets of {1, 2, 3} were printed, including the empty subset.

Viva-Voce Questions:

- How many subsets does a set of n elements have?
- What is backtracking?
- Why is the last element removed after the recursive call?

Experiment No. 15.2 - All arrangements of a, b, c

Experiment Title:

All arrangements of a, b, c

Aim:

To print all arrangements of three letters using a used[] array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Backtracking builds a solution one choice at a time. After a complete answer is printed, the last choice is undone so that the next choice can be tried. For subsets, each element is either skipped or included, giving 2^n subsets. For arrangements (permutations), a used[] array marks letters already placed, giving n! arrangements.

Algorithm / Procedure:

1. Keep a used[] array and a result[] array; pos is the position being filled.
2. Base case: when pos == length, print the arrangement.
3. For each letter that is not used: mark it used, place it at result[pos], recurse with pos + 1.
4. After the call returns, unmark the letter (backtrack).

Program / Queries / Commands:

```
public class Main {
    static void arrange(char[] letters, boolean[] used, char[] result,
            int pos) {
        if (pos == letters.length) {
            System.out.println(new String(result));
            return;
        }

        for (int i = 0; i < letters.length; i++) {
            if (!used[i]) {
                used[i] = true;
                result[pos] = letters[i];
                arrange(letters, used, result, pos + 1);
                used[i] = false;
            }
        }
    }
```

```
    public static void main(String[] args) {
        char[] letters = {'a', 'b', 'c'};
        boolean[] used = new boolean[letters.length];
        char[] result = new char[letters.length];
        arrange(letters, used, result, 0);
    }
}
```

Input:

Letters a, b, c.

Output:

```
abc
acb
bac
bca
cab
cba
```

Time / Space Complexity: Time O(n * n!); Space O(n).

Result:

The program was compiled and executed successfully. All 6 arrangements of a, b and c were printed.

Viva-Voce Questions:

- How many arrangements do n distinct letters have?
- What is the purpose of the used[] array?
- Why is used[i] set back to false?

Experiment No. 15.3 - All arrangements of x, y

Experiment Title:

All arrangements of x, y

Aim:

To print all arrangements of two letters using a used[] array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Backtracking builds a solution one choice at a time. After a complete answer is printed, the last choice is undone so that the next choice can be tried. For subsets, each element is either skipped or included, giving 2^n subsets. For arrangements (permutations), a used[] array marks letters already placed, giving n! arrangements.

Algorithm / Procedure:

1. Keep a used[] array and a result[] array; pos is the position being filled.
2. Base case: when pos == length, print the arrangement.
3. For each letter that is not used: mark it used, place it at result[pos], recurse with pos + 1.
4. After the call returns, unmark the letter (backtrack).

Program / Queries / Commands:

```
public class Main {
    static void arrange(char[] letters, boolean[] used, char[] result,
            int pos) {
        if (pos == letters.length) {
            System.out.println(new String(result));
            return;
        }

        for (int i = 0; i < letters.length; i++) {
            if (!used[i]) {
                used[i] = true;
                result[pos] = letters[i];
                arrange(letters, used, result, pos + 1);
                used[i] = false;
            }
        }
    }
```

```
    public static void main(String[] args) {
        char[] letters = {'x', 'y'};
        boolean[] used = new boolean[letters.length];
        char[] result = new char[letters.length];
        arrange(letters, used, result, 0);
    }
}
```

Input:

Letters x, y.

Output:

```
xy
yx
```

Time / Space Complexity: Time O(n * n!); Space O(n).

Result:

The program was compiled and executed successfully. Both arrangements of x and y were printed.

Viva-Voce Questions:

- How many arrangements exist for 2 letters?
- Is this recursion or iteration?
- How does the program change for more letters?